Empirical Software Engineering manuscript No.
(will be inserted by the editor)

What Carries Architectural Signal? A Code-to-Architecture Mapping Study

Rakhshanda Jabeen · **Morgan Ericsson** · **Yuyao Duan**

Received: date / Accepted: date

Local review draft, 5 October 2026. Full-dataset results integrated; manuscript and review artifacts remain unpublished.

Abstract Assigning source files to reference architecture modules supports conformance checking and system understanding. Identifiers, code content, static dependencies, and directory organization inform this mapping, but their usefulness depends on how they are represented and processed. We investigate the contribution of file representations, folder and member information, and dependency neighbors, defined as files connected by static dependencies. We study ten Java, C, and C++ systems with limited labels, comparing Word2Vec, Node2Vec, and UniXcoder representations with feature-only and graph-based predictors. We vary file, member, and folder graph configurations and compare predictors with and without dependency neighbors, aggregated along the dependency direction or in both directions. Representations that encode dependencies provide strong feature-only baselines, while folder nodes yield the largest and most consistent gains among graph configurations. However, these gains largely reflect information available through a directory rule and do not consistently improve predictions for files that depart from it. With file representations held fixed, dependency neighbors raise the median score of a multilayer perceptron (MLP) receiving each file's representation and the mean representation of its neighbors, of a Graph Attention Network (GAT), and of a Heterogeneous Graph Transformer (HGT) when aggregated in both directions, with the smallest gain for HGT. Aggregating only along the dependency direction lowers median macro-F1 with Node2Vec, and the better direction depends on the system and not on the predictor or the representation. Useful dependency information therefore does not by itself determine whether neighbor aggregation helps. Evaluations should distinguish information already encoded in file representations from the contribution of neighbor access and its direction, and assess directory-departing files separately from aggregate performance.

Keywords Software Architecture · Heterogeneous Graph Neural Networks · Graph Transformers · Representation Learning

Rakhshanda Jabeen
Electrolux Professional AB, Ljungby, Sweden
Linnaeus University, Växjö, Sweden
E-mail: rakhshanda.jabeen@electroluxprofessional.com

Morgan Ericsson
Linnaeus University, Växjö, Sweden
E-mail: morgan.ericsson@lnu.se

Yuyao Duan
Linnaeus University, Växjö, Sweden
E-mail: yuyaoduan4@gmail.com

1 Introduction

Maintaining a software architecture requires knowing which implementation elements belong to its modules. That correspondence can become difficult to maintain as a system grows: new files introduce responsibilities and dependencies, while architectural documentation may lag behind the implementation. Code-to-architecture (C2A) mapping addresses this problem by assigning source files to the modules of a reference architecture. The resulting mapping supports reflexion modeling, which compares implementation dependencies with architectural relations to identify divergence [37]. It therefore connects the modular decomposition used to reason about a system [47, 9] to the code in which that decomposition is realized.

Several sources of evidence can support a mapping. Identifiers express domain concepts and implementation roles. Contextual code representations capture information from declarations and operations. Static dependencies locate a file among other files with which it interacts, while directories

record how developers organize the project [63, 6, 15]. These sources need not agree: a utility may depend on several modules, terminology may be shared across responsibilities, and a directory may contain files assigned to different architectural modules.

The usefulness of an information source also depends on how it enters a predictor. A dependency embedding can encode graph structure before a classifier sees a file. Propagating over dependencies then supplies a second form of access to related files, with its own assumptions about direction, relation types, and aggregation. A gain from the embedding does not establish that this additional propagation will help. Similarly, a folder-aware model can improve overall agreement with a reference architecture while losing correct predictions for files that depart from a directory-based assignment. These distinctions motivate studying both the information supplied to a model and the changes caused by the way it processes that information.

Prior architecture recovery and mapping work provides several ways to represent these sources. NEGAR learns structural representations using Node2Vec random walks on the dependency graph [6]; GAER used a graph autoencoder with Word2Vec identifier embeddings for recovery [22]; prior mapping work uses lexical and organizational features with graph convolutional networks [21]. Later work compares random walk-based methods with graph attention and diffusion-based rewiring and examines alternative uses of dependencies and folders [23]. These studies generally fix the representation chosen for each source, so it remains unclear which signal provides the strongest architectural evidence and how it should be represented.

We study this question through C2A mapping, a prerequisite for reflexion modeling [37]. We formulate C2A mapping as a classification problem in which each file is assigned to one module of the reference architecture. We work in a low-label setting, where about 5% of the files are already mapped, and the model infers the remaining assignments. The model is trained on the labeled files alone. This setting allows us to examine how file representations and graph relations contribute under limited supervision.

Our evaluation uses ten Java, C, and C++ systems and covers lexical Word2Vec, structural Node2Vec, and contextual UniXcoder representations. These serve as inputs to a graph-free neural baseline, to a Heterogeneous Graph Transformer (HGT) [20], and, in the dependency comparison, to a neighbor MLP and a Graph Attention Network (GAT) [60]. The heterogeneous configurations contain file nodes and, where applicable, folder nodes and one aggregate member node per file. They vary both the context available to a file and where feature content is stored. We compare these file representations and complement the configuration comparisons with controlled comparisons that change one design choice while holding the features, labeled splits, initialization, and training budgets fixed within each comparison.

The configuration results motivate two complementary analyses. A directory analysis distinguishes errors corrected by a model from previously correct predictions that it makes incorrect. A dependency analysis compares ways of using dependency neighbors on all ten systems, switching neighbors on and off and aggregating them along the dependency direction or in both directions.

We address three research questions:

- **RQ1.** How does mapping performance vary across lexical, structural and contextual file representations?
- **RQ2.** How does mapping performance change when file dependencies, member nodes, and folder nodes are added to each representation?
- **RQ3.** How do the predictor and dependency direction affect the benefit of neighbors?

The results show that dependency information supports C2A mapping through both file representations and neighbor context, but the benefit of neighbor context depends on how it is processed. Folder organization yields the largest gains, largely reflecting information already available through a directory rule. Aggregate improvements can conceal regressions on individual files. These findings clarify how input signals and their use by a predictor affect C2A mapping across the nine systems studied[1].

2 Related Work

Architecture recovery and code-to-architecture mapping both infer a system's modular structure from implementation artifacts, relying on evidence that includes static dependencies, code semantics, and directory organization, among other sources. Recovery produces a decomposition without a reference architecture, while mapping assigns entities to the modules of a given one. We evaluate C2A mapping, and draw on recovery research for representations of dependencies, lexical content and project organization. We organize this work by information source while keeping the two evaluation tasks distinct.

Architecture recovery techniques have historically been built around one dominant signal. Dependency-based methods treat the system as a graph and partition it to maximize modularization quality, using search-based optimization [30, 34, 48, 31, 18] or clustering over dependency-derived similarity [39, 40, 1, 58, 36, 49]. These perform well when dependencies align with architectural boundaries, but recovery quality is sensitive to how the graph is constructed and how relations are weighted, since shared utilities and

[1] Code and data are available at `https://github.com/sse-lnu/hetmap`.

layered interactions introduce cross-module edges. Semantic methods instead cluster artifacts by identifiers and comments in a latent space learned from source code [14, 11, 53] or from lexical cues [27], and directory-based methods use file names and paths as an architectural signal [2]. Non-structural signals can be noisy in isolation because reused terminology and utility directories mix entities from many modules. Hybrid methods therefore combine sources through weighted similarity fusion and many-objective remodularization [33, 51, 35, 55], consensus clustering [50], semantic-guided propagation [62], or weighted graphs with community detection [46, 63], using different mechanisms to combine the available evidence.

Mapping shows the same pattern from a different starting point. Early work framed the assignment of unmapped entities as orphan adoption [59], and HuGMe introduced attraction functions to recommend mappings between entities and modules [7, 8]. Subsequent methods varied the signal and the learner: information-retrieval attraction based on textual similarity [4], supervised classifiers for orphan adoption [3], and a semi-supervised Naive Bayes approach combining lexical and dependency-derived information through Concrete Dependency Abstraction [42, 44]. Interactive and text-based approaches followed, comparing text classifiers for module assignment [12], recommending mappings from natural-language architecture descriptions [56], and integrating package- and class-level granularity [57]. More recent work frames method-level mapping as text classification over TF-IDF, GloVe, and LLM representations [26], examines how textual data properties condition embedding-based classification performance [25], and extracts architectural concepts from documentation and code [13]; LLM-based chain-of-thought prompting has also been used to map implementation units to a reference architecture [54], though such prompting-based approaches face scalability challenges on large codebases given context-size limits and the cost of repeatedly processing implementation artifacts.

Representation-learning approaches differ in both the information they encode and the operations used to recover or predict modules. NEGAR clusters node2vec embeddings from the dependency graph alone, with no node semantics [6]. Jabeen et al. [23] build on this by adding folder organization and comparing node2vec, meta-path walks, and graph attention with and without diffusion rewiring, without lexical features. GAER and Jabeen et al. [22, 24] instead fix folder and identifier tokens into one encoding, Word2Vec-based in the former and contrastively trained with edge attributes in the latter, without comparing it to alternatives. For mapping, prior work evaluated graph neural networks on homogeneous and dependency-type-aware graphs [21], without folder hierarchies, containment, member-level artifacts, or separating graph contribution from feature contribution. These approaches motivate our distinction between representing an information source and using it during prediction. Our configuration comparisons span lexical, structural and contextual inputs, but retain differences between their training procedures and feature content. We therefore complement them with matched dependency controls and directory-error analyses. These controls assess specific design choices; they do not turn the broader configuration comparison into an isolated ranking of information sources.

3 Background

Software artifacts are naturally relational and multi-typed. A system contains folders, source files or classes, and finer-grained program elements such as methods and fields, connected through relations that include folder containment, file-to-member composition, and static code-level dependencies such as calls, inheritance, type usage, and field access.

To preserve these distinctions, we represent each software system as a heterogeneous software graph $G = (V, E, \mathcal{A}, \mathcal{R})$. Here, V denotes the set of software artifact nodes and E denotes the set of relations between them. Each node $v \in V$ is assigned a node type through the mapping function $\tau(v) : V \to \mathcal{A}$, where $\mathcal{A}$ contains folder, file, and aggregate member types in our most detailed configuration. Similarly, each edge $e \in E$ is assigned a relation type through $\phi(e) : E \to \mathcal{R}$, capturing folder hierarchy, the association between folders and classes, the association between classes and finer-grained program elements, and static code dependencies such as method calls, inheritance, and imports.

In this graph, the meaning of a relation is not characterized by the relation type alone. It also depends on the types of the source and target nodes that the edge connects. Therefore, for an edge $e_{s,t}$ from source node s to target node t, we describe the typed interaction as a meta-relation $\langle \tau(s), \phi(e_{s,t}), \tau(t) \rangle$. For example, a $\langle$folder, *contains*, file$\rangle$ relation captures project organization, whereas a $\langle$file, *Import*, file$\rangle$ relation captures code-level coupling. Thus, the same graph can preserve both structural organization and implementation dependencies without treating them as identical relations. A homogeneous graph, with one node type and one relation type, discards this distinction, and RQ3 uses it to test whether the typed, directed form matters.

HGT [20] exploits such meta-relations during message passing by conditioning attention and message transformation on the source node type, edge type, and target node type. GAT [60] also uses attention-based message passing, but its standard formulation does not explicitly distinguish node or relation types. HGT can also operate on a graph with a single node type and a single relation type, although this does not make its attention mechanism equivalent to that of GAT, and RQ3 compares both on such a graph. The initial node features determine what information is available to these models, so graph structure and feature representation describe different

parts of a mapping model, even when both are derived from dependencies.

A lexical embedding trained on identifier tokens captures naming conventions but not a file's position within the dependency structure or the syntactic and behavioral content of its implementation. An embedding trained directly on the dependency graph, such as Node2Vec [16], instead encodes a file's structural neighborhood without reference to its content, and such embeddings have been shown to capture dependency-based cohesion effectively in architecture recovery [6, 23].

Pretrained code encoders such as UniXcoder [17] address the content gap left by lexical features by learning general-purpose representations of source code from large corpora, but their pretraining objective is not specific to architectural grouping: two files can be lexically and syntactically similar while belonging to different modules. Contrastive learning offers a way to adapt such an encoder to a task-specific notion of similarity by training the model to place designated positive pairs closer in the embedding space than unrelated in-batch examples, typically through an InfoNCE objective [45]. Using positive pairs derived from dependencies encourages the embedding space to reflect dependency proximity. It does not guarantee that this proximity coincides with the reference modules.

4 Methodology

This section presents the methodology for studying how architectural signals and graph design choices affect code-to-architecture mapping. We formulate the mapping task as node classification over a software graph $G = (V, E, \mathcal{A}, \mathcal{R})$, where $\mathcal{A}$ and $\mathcal{R}$ denote the sets of node and relation types. Let $V_f \subseteq V$ be the set of file nodes, which are the targets of the mapping task. A small subset $L \subset V_f$ is labeled with known architectural module assignments, and the remaining file nodes $U = V_f \setminus L$ are unlabeled. The objective is to learn an encoder:

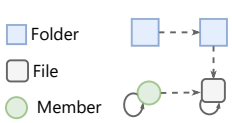
$$Z = f_\theta(G), \qquad Z \in \mathbb{R}^{|V_f| \times d}, \tag{1}$$

that maps each source file $i \in V_f$ to a d-dimensional embedding z_i, where θ denotes the learnable parameters of the representation-learning method. These file embeddings are used to predict architectural modules and to infer the labels of the nodes in U.

We first describe the graphs and the structural, lexical, and contextual features used to initialize their nodes. We then present the neural baseline MLP and the HGT and GAT encoders. RQ1 varies the file representation, using the MLP. RQ2 varies the graph configuration for each representation and compares the results with a directory baseline described in Section 5.2.1. RQ3 varies the predictor, comparing the MLP with GAT and HGT message passing.

4.1 Heterogeneous Graph Construction

We model the software system as a heterogeneous graph whose nodes represent software artifacts and edges represent organizational, containment, and dependency relations. The graph is built from three sources of information: the folder structure of the project, the containment of program elements within a file, and static dependencies extracted from source code. We preserve these sources as distinct node and relation types rather than collapsing them into a single homogeneous graph. Figure 1 illustrates the general schema of the heterogeneous software graph and the meta-relations used in our model.



(a) Software graph schema

Folder	*parent_of*	Folder
Folder	*contains*	File
File	*depends_on*	File
Member	*belong_to*	File
Member	*depends_on*	Member

(b) The meta relations

Fig. 1: Schema and meta-relations of the heterogeneous software graph. The figure shows the most expressive representation, with folder, file, and member nodes. Simpler variants remove selected node types or aggregate member-level dependencies at the file level.

4.1.1 Node Types

We use three node types: folder, file, and member. Folder nodes represent directory levels in the project structure. File nodes represent source files and are the target nodes for code-to-architecture mapping. Member nodes represent member-level program information associated with a file, including methods, fields, constructors, and variables.

We use one summarized *Member* node per file rather than representing each method, field, constructor, or variable as a separate node. The member node aggregates lexical and dependency information from the members of its associated file, retaining member-level information while avoiding the large and sparse graphs produced by fully fine-grained representations.

4.1.2 Relation Types

The graph contains structural relations, including organization, containment, and dependency. Logical components are often implemented through namespaces or directory structures that group source code providing a particular functionality, and such structures reflect how developers organize files [52, 15]. To preserve this information, each directory level is represented as a separate folder node. Folder nodes are connected through *parent_of* edges, forming hierarchical chains such as `app` → `app/ui` → `app/ui/views`. File

Table 1: Structural and static dependency relation types used in the software graph.

Relation type	Description
parent_of	Folder contains a subfolder in the directory.
contains	Folder contains a source file.
belongs_to	Member-level program element belongs to a file.
depends_on	Static dependency between files or members.

nodes are linked to their folder through *contains* edges, and each member node is connected to its enclosing file through a *belongs_to* edge.

Static dependencies provide direct evidence of how entities interact and are a central source of information for architecture mapping and recovery [6, 29]. The static dependencies were extracted with the Depends tool [41], which analyzes the syntactic structure of the projects studied in this paper and identifies entities together with several categories of static relations. We merge all dependency types between the same pair of node types into a single *depends_on* relation, drop duplicate edges, and remove edges whose endpoints belong to the same file. Table 1 lists the structural relations and the dependency relation used in the graph.

4.1.3 Graph Variants

Folder organization, member-level information, and static dependencies are available simultaneously in the complete graph, so their contributions cannot be determined from the full configuration alone. We therefore construct four graph variants that vary the presence of folder and member nodes while retaining dependency information at the available granularity. Figure 2 provides a visual overview.

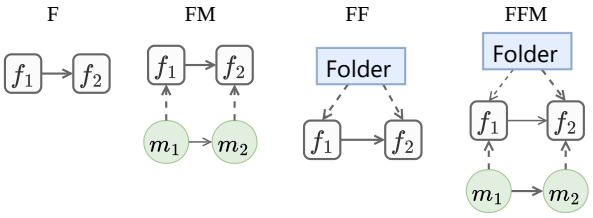


Fig. 2: Graph configurations: files f_i are the prediction targets; each m_i aggregates the members of file f_i. Solid arrows show dependencies and dashed arrows show meta-relations. The panels omit relation types, additional dependency edges, and folder ancestry; Figure 1 gives the typed schema. Feature construction also depends on the configuration, as described in Section 4.2.

We classify dependencies according to their resolved endpoints. A dependency is file-level if at least one endpoint is a file; this includes file–file, file–member, and member–file interactions. It is member-level if both endpoints are members. FFM is the complete graph and keeps these dependency edges separate from containment relations, preserving their endpoint types. FM removes folder nodes and relations while retaining files and members. FF removes member nodes instead, and F removes both folder and member nodes and serves as the file-level dependency baseline. In FF and F, each member endpoint is replaced by its enclosing file, so all remaining dependencies connect file nodes.

4.2 Node Feature Initialization

We initialize node features based on the information source used in each experimental setting. Folder nodes are always initialized from binary folder-path tokens, regardless of which strategy is used elsewhere in the graph. File and member nodes are initialized using one of three strategies: lexical embeddings from Word2Vec, structural embeddings from Node2Vec, or contextual embeddings from UniXcoder, used zero-shot or fine-tuned with dependencies as described below.

4.2.1 Folder Node Features

Folder nodes are encoded using a binary bag-of-words representation over folder tokens. The folder tokens are derived from directory segments after removing common project-root tokens. Each folder node is represented by the tokens along its path in the directory hierarchy. For example, a folder corresponding to `app/ui/views` carries the tokens `app`, `ui`, and `views`. This allows deeper folder nodes to retain information about their hierarchical context.

4.2.2 Word2Vec (W2V): Lexical Features

Source code identifiers often encode domain concepts and implementation roles, making them useful cues for mapping entities to architectural modules [63]. We use Word2Vec embeddings following the approach of Jabeen et al. [22] for lexical input. Tokens are extracted from file names and member-level program elements, including methods, fields, constructors, and variables. We normalize identifiers by splitting camelCase, PascalCase, and snake_case forms, converting tokens to lowercase, and removing stopwords and non-alphabetic tokens. The remaining tokens are represented using 128-dimensional skip-gram Word2Vec embeddings [32], trained for 20 epochs with a context window of five. Each node representation is computed by averaging the embeddings of its associated tokens.

The graph variant determines where these features are stored. In FFM and FM, the file node contains file-name tokens and the member node contains identifiers from the file's program elements. FF and F have no member nodes, so member identifiers are included in the file representation

instead, and the graph-free MLP uses the same file vector as F and FF. Member identifiers therefore remain available in every variant.

4.2.3 Node2Vec (N2V): Structural Features

Chen et al. [6] observe that files in the same architectural module can occupy a dense local dependency region without being directly connected. Random walks can capture such longer-range dependency structure [23]. We use Node2Vec's [16] random-walk embeddings as a structural initialization and merge all static dependency types into a single edge type before generating random walks. Its biased random walks use parameters p and q to balance local and exploratory walks. The resulting embeddings capture structural similarity from dependency position without using identifier text or folder location.

In FM and FFM, each aggregate member node receives the Node2Vec embedding of its corresponding file.

4.2.4 UniXcoder: Contextual Features

For contextual initialization, we use UniXcoder [17], a pretrained code transformer. The input text is obtained through syntax-tree-guided extraction. Source files are parsed with Tree-sitter [5], using the Java, C, or C++ grammar that matches each file, and the syntax tree determines which source fragments are passed to the encoder; the tree itself is not an input to the model. Extraction produces two views: a file-level view containing type declarations, fields or global variables, and constructor signatures with short body excerpts where applicable, and a member-level view containing the file's methods or functions. Each file receives one vector per view. The member-level text is split into overlapping chunks that are mean-pooled, so there is one member-level vector per file and not one per method.

The zero-shot variant ($\mathrm{UXC_{ZS}}$) encodes this text with the pretrained weights. The dependency-guided variant ($\mathrm{UXC_{FT}}$) fine-tunes the encoder separately for each system with a contrastive objective [45], using only the file-level views; module labels do not enter the loss. The fine-tuned model then encodes both views, so the member-level vectors come from a model that was not fine-tuned on member text. Because shallow message passing over raw dependencies struggles to relate files that share neighbors but are not directly connected [23], positive pairs are drawn from the two-hop neighborhood, treating dependencies as undirected: for each anchor file, two-hop candidates are ranked by the number of shared one-hop neighbors, two pairs per file are sampled from the top eight, and files without a two-hop candidate fall back to a direct neighbor. Other pairs in the same batch serve as negatives, except that in-batch files connected by a dependency are masked so that they act as neither positive nor negative. Only the upper four encoder layers are updated, while the token embeddings and the lower eight layers stay frozen.

Member information therefore enters the model differently for each representation. W2V moves member identifiers from the file vector to the member node when member nodes are present, UniXcoder provides member text only through the member node, and N2V gives the member node the file's own vector. Moving from F to FM or FFM thus changes how information enters the model as well as the graph, and the comparisons across graph variants are interpreted for each representation separately.

4.3 Mapping Models

All models map the final file representation to module logits with a linear layer and are trained by minimizing categorical cross-entropy over the labeled file nodes. They differ in how that file representation is computed.

4.3.1 Multilayer Perceptron (MLP)

The MLP predicts module logits directly from a file-level input vector. Each hidden layer computes:

$$h_i^{(k+1)} = \mathrm{ReLU}\big(W^{(k)} h_i^{(k)} + b^{(k)}\big), \tag{2}$$

with $h_i^{(0)} = x_i$ the file feature vector and $W^{(k)}, b^{(k)}$ the learnable parameters of layer k, and ReLU is the rectified linear unit [38]. After the final hidden layer, a linear layer maps the resulting representation to module logits. The input x_i contains the single representation being evaluated.

4.3.2 Heterogeneous Graph Transformer (HGT)

Node types have different feature spaces depending on the graph variant and the initialization strategy, so HGT first maps each node type into a shared hidden space using a type-specific linear transformation followed by ReLU. One HGT layer [20] then updates node representations through multi-head attention over their neighbors. For a target node t and a source node s connected by an edge e, the layer computes a message $M(s,e,t)$ and an attention weight $\alpha(s,e,t)$, and aggregates them as:

$$m_t = \sum_{s \in \mathcal{N}(t)} \alpha(s,e,t)\, M(s,e,t), \tag{3}$$

$$M(s,e,t) = V_{\tau(s)}\, h_s\, W^{\mathrm{MSG}}_{\phi(e)},$$

$$\alpha(s,e,t) = \underset{s \in \mathcal{N}(t)}{\mathrm{softmax}} \frac{\big(K_{\tau(s)} h_s\big)\, W^{\mathrm{ATT}}_{\phi(e)} \big(Q_{\tau(t)} h_t\big)^{\top}\, \mu_{\langle \tau(s), \phi(e), \tau(t) \rangle}}{\sqrt{d}}.$$

where $\tau(\cdot)$ is the node type, $\phi(e)$ is the edge type, V, K, and Q are type-specific linear transformations, W^{MSG} and W^{ATT} are edge-type-specific matrices, μ is a learned prior for each meta-relation, d is the head dimension, and attention-head indices are omitted for brevity. These parameters allow a

contains edge from a folder and a *depends_on* edge from a dependent file to receive different attention weights. The aggregated message is combined with the node's projected input $h_t^{(0)}$ through a residual connection [19] followed by layer normalization:

$$h_t^{(1)} = \text{LayerNorm}\left(m_t + h_t^{(0)}\right). \quad (4)$$

Folder and member nodes, when present, contribute to file representations through message passing.

4.3.3 Graph Attention Network (GAT)

GAT [60] aggregates neighbor representations with a single attention mechanism shared by all neighbors, without the node-type- or edge-type-specific parameters of HGT, and concatenates the outputs of several attention heads. Like HGT, it first projects the input vector and then combines the aggregated message with the projected input through a residual connection followed by layer normalization, so the two models differ only in how neighbors are aggregated. GAT adds no self-loops; the residual connection keeps the node's own representation.

5 Experiments

This section describes the benchmark systems, the experimental configuration, the directory-based reference predictor, and the evaluation procedure used to answer the three research questions.

5.1 Benchmark Systems

We evaluate the HGT-based model on ten open-source systems with published ground-truth (GT) mappings, which have been used in prior work on architecture recovery and C2A mapping [22, 44]. They differ in programming language, domain, size, and architectural granularity. We use published mappings because reliable C2A mappings require substantial manual effort and expert validation [15], and because they allow comparison with previous studies.

Table 2 summarizes the system and graph properties. File-level dependency homophily $\mathcal{H}$ is the proportion of unique directed file pairs connected by a static dependency whose source and target files fall within the same GT module, from 0 (every dependency crosses a module boundary) to 1 (every dependency stays within a module). The mean number of outgoing typed dependency edges per file, $\langle k \rangle$, differs from homophily in counting different dependency types between the same file pair separately.

The Java systems come from the benchmark of Olsson et al. [44]. JabRef and TeamMates (T.Mates) were studied at the SAEroCon workshop in 2016 and 2017, where system experts provided the architectures and mappings[2]. Lucene was studied by Lenhard et al. [28], and the Commons Imaging (C.Img) mapping was contributed as a test case for s4rdm3x [43] [3].

Table 2: Subject systems and their file-level dependency graph properties. M: number of modules in the GT architecture; $|E|$: number of dependency edges, counted separately for each dependency type; H: dependency homophily; $\langle k \rangle$: average node degree, $|E|/|\text{Files}|$.

	System	Version	LoC	M	Files	$\lvert E\rvert$	H	$\langle k \rangle$
Java	A.UML	v0.24	n/a	14	766	9 491	0.55	12.39
	C.Img	v1.0a2	18k	21	329	4 561	0.61	13.86
	JabRef	v3.7	59k	6	1 013	16 394	0.56	16.18
	Lucene	r1075001	36k	7	1 054	18 595	0.68	17.64
	SH-3D	r002382	35k	9	167	3 912	0.36	23.43
	T.Mates	v5.11	55k	15	778	17 818	0.34	22.90
C	Bash	4.2	115k	14	292	1 222	0.58	4.18
	HDF	0e196f	153k	9	153	1 239	0.35	8.10
C++	HDC	46ff87	26k	11	207	1 117	0.63	5.40
	Chrome	svn-171054	11M	69	18 343	284 435	0.58	15.51

For Bash, we use the mapping curated by Garcia et al. [15], who recovered an initial decomposition from documentation and source code and refined it with system experts through a certification process. The same work provides the Chromium mapping, developed with developers over approximately two years [29].

The HDF and HDC mappings were developed with industry partners and are part of the SARIF benchmark of Zhang et al. [63], available in its replication package[4].

5.2 Experimental Configurations

We repeat each experiment 100 times with a different initial labeled set. In each run, we randomly sample 5% of the file nodes, with a minimum of 30 and a maximum of 200 files, and add one file from every GT module absent from the sample so that all modules are represented. All compared configurations use the same initial labeled set within a run, which allows paired comparisons, and the repetition measures sensitivity to which files are labeled. The same 100 splits are used in every experiment.

Input representations. The W2V embeddings are 128-dimensional vectors trained for 20 epochs with a context window of five on the file names and identifiers of each system. N2V learns 128-dimensional vectors from random walks on the undirected file-level dependency graph (walk length 30, 200 walks per file, $p = q = 1$, window 15, five skip-gram epochs and five negative samples). Chromium uses 100 walks

[2] https://github.com/sebastianherold/SAEroConRepo

[3] Java labels are available at: https://github.com/h0bb3/s4rdm3x

[4] https://github.com/anonymous2f4a9d/SARIF_FSE23

per file and one negative sample. The N2V vectors are computed once per system and reused across runs. For UXC_{FT}, UniXcoder is fine-tuned with an InfoNCE temperature of 0.05, AdamW with a learning rate of 2×10^{-5}, a batch size of eight, and two epochs with a fixed seed. Both UniXcoder variants encode file inputs with a 512-token limit and member inputs in overlapping 256-token windows with a stride of 128, mean-pooled before normalization, producing 768-dimensional features. Graph structures follow Section 4.1.

Training models. All mapping models are trained with Adam at a learning rate of 0.001 for 200 epochs on the labeled files only, without self-training. Weights are initialized from a seed set by the split, and the neighbor settings of a model share them. GAT and HGT use one layer, 256 hidden dimensions, and four attention heads, and the MLP uses two hidden layers with 256 dimensions.

RQ1 trains the MLP on each of the four representations, and RQ2 trains HGT on the F, FM, FF, and FFM graphs (Figure 2) for W2V, N2V, and UXC_{FT}. RQ3 reuses this setup on the file graph for all four representations and varies the predictor (MLP, GAT, HGT) and the neighbor information: *off*, *directed* (from the source to the target of each dependency), or *undirected* (each dependency also connects its files in the reverse direction). The MLP receives its own vector concatenated with the mean of the vectors of the neighbors it receives from, and only its own vector when neighbors are off. The MLP without neighbors is the RQ1 classifier and HGT with directed neighbors is the F configuration of RQ2, which gives 45,000 runs in total.

5.2.1 Directory Hierarchy Baseline

To assess how much folder-based configurations exploit files in the same directory sharing a module, we use a rule-based predictor based on the directory hierarchy and the initial labeled files. For each unlabeled file, it assigns the most common module among labeled files in the same directory. If none are present, it searches the directory subtree and then the subtrees of its parent directories until it finds labeled files. Ties are resolved by the module's frequency in the complete initial labeled set and then alphabetically. If no labeled files are found, the predictor assigns the most common module in the complete initial labeled set. It uses the directory hierarchy, but not directory-name text, source code, embeddings, dependencies, or test labels.

We apply the predictor to all 100 initial labeled sets and compare it with HGT using UXC_{FT} features on the directed FM and FFM graphs; the F and FF graphs are not included. For each system, we report the accuracy of the predictor, FM, and FFM. For the files that the predictor assigns incorrectly, we also count, per file and split, the correct FM predictions that FFM loses and the incorrect FM predictions that it corrects.

5.3 Performance Evaluation

The benchmark systems have imbalanced module distributions, with some modules containing many files and others only a few. Accuracy can therefore overstate performance by giving disproportionate weight to the largest modules, so we evaluate mapping performance with the F1 score, the harmonic mean of precision (the proportion of predicted module assignments that are correct) and recall (the proportion of true assignments that are recovered). We report macro- and micro-averaged F1. Macro-F1 is computed for each architectural module and then averaged, giving every module equal weight, so it is sensitive to small modules. Micro-F1 aggregates predictions across all files before computing F1 and gives greater weight to larger modules. Reporting both distinguishes class-balanced from overall file-level performance.

6 Results

This section reports the results for code-to-architecture mapping across ten subject systems. RQ1 compares lexical, structural, and contextual file representations using a feature-only classifier. RQ2 examines how mapping performance changes when these representations initialize HGT with file, member, and folder context, and analyzes errors relative to directory-based predictions. RQ3 uses controlled experiments to examine how the predictor and the direction of dependency neighbors affect the benefit of neighbors. For RQ1 and RQ2, we report median macro- and micro-F1 scores over 100 runs per system and configuration; for RQ3, we report changes in F1 points between paired conditions over 100 splits. Every test file is scored, regardless of prediction confidence.

Within each run or split, compared configurations use the same randomly sampled initial labeled set. For each system and metric, we use the two-sided Wilcoxon signed-rank test [61], pairing runs by their shared split. We apply Holm's correction within each comparison family, separately for macro-F1 and micro-F1, at $\alpha = 0.05$. We report differences in median performance in F1 percentage points and, for RQ1 and RQ2, Cliff's δ [10], classified by magnitude as negligible (N, $|\delta| < 0.147$), small (S, $0.147 \leq |\delta| < 0.33$), medium (M, $0.33 \leq |\delta| < 0.474$), or large (L, $|\delta| \geq 0.474$).

RQ1: Comparing Lexical, Structural, and Contextual File Representations

RQ1 examines how much predictive information each file representation provides before additional graph-based message passing. Table 3 reports the median macro-F1 and micro-F1 scores obtained with lexical W2V, structural N2V, and UniXcoder variants: UXC_{ZS} and UXC_{FT}. Each representation is evaluated using the graph-free MLP. This classifier

performs no message passing, although N2V encodes dependency topology directly and UXC_{FT} incorporates dependency information through its fine-tuning procedure. Table 4 summarizes the pairwise differences across all ten systems, with the number of systems favoring each method and the number of Holm-significant differences in each direction.

Table 3: Median macro-F1/micro-F1 scores (%) across 100 runs for the feature-only MLP using W2V, N2V, and contextual representation variants:UXC_{ZS} and UXC_{FT}. Highlighted values indicate the highest score for each system and metric.

System	W2V	N2V	UXC_{ZS}	UXC_{FT}
A.UML	34 / 57	54 / 72	36 / 60	46 / 69
C.Img	23 / 39	71 / 80	33 / 52	42 / 62
JabRef	44 / 78	40 / 74	39 / 76	43 / 79
Lucene	57 / 71	66 / 79	54 / 66	63 / 76
SH-3D	22 / 41	43 / 58	48 / 64	47 / 67
T.Mates	30 / 55	41 / 68	37 / 68	51 / 79
Bash	7 / 29	42 / 61	23 / 42	27 / 49
HDF	8 / 22	53 / 57	42 / 44	46 / 48
Chrome	14 / 48	41 / 71	14 / 48	31 / 66
HDC	33 / 53	43 / 69	30 / 51	46 / 70

Since the subject systems contain small and imbalanced architectural modules, macro-F1 is generally lower and more variable than micro-F1. N2V achieves the highest median macro- and micro-F1 scores on six systems. On the other four systems, UXC_{FT} obtains the highest median micro-F1, while the highest median macro-F1 is held by UXC_{FT} on T.Mates and HDC, W2V on JabRef, and UXC_{ZS} on SH-3D.

We first compare W2V and UXC_{ZS}, the two representations that do not encode dependencies. They do not separate consistently: the significant differences fall in both directions, although UXC_{ZS} is favored on more systems. On macro-F1, UXC_{ZS} is significantly stronger on A.UML (medium effect), C.Img, SH-3D, T.Mates, Bash, and HDF (large effects), whereas W2V is significantly stronger on JabRef (large effect), Lucene and Chrome (medium effects), and HDC (small effect). On micro-F1, UXC_{ZS} is significantly stronger on the same six systems, and W2V is significantly stronger on JabRef and Lucene (large effects) and HDC (small effect); the difference in Chrome is not significant.

Fine-tuning significantly improves UniXcoder over UXC_{ZS} on micro-F1 for all ten systems and on macro-F1 for nine, predominantly with large effects. The effects are medium on HDF (both metrics) and on SH-3D micro-F1; the macro-F1 difference on SH-3D is not significant.

Both representations encoding dependency information are generally stronger than the lexical representation. N2V significantly outperforms W2V on both metrics for nine systems, with JabRef as the exception, where W2V is significantly stronger. Similarly, UXC_{FT} significantly outperforms W2V on both metrics for nine systems; on JabRef, the difference is not significant on either metric.

Table 4: Summary of pairwise differences in F1 points across ten systems. Δ is the median difference (first method minus second); +ve counts systems favoring the first method; $\uparrow / \downarrow$ gives the number of significant positive/negative differences.

	Macro F1			Micro F1		
Comparison	Δ	+ve	$\uparrow / \downarrow$	Δ	+ve	$\uparrow / \downarrow$
$UXC_{ZS}-W2V$	+5.2	6	6/4	+7.5	6	6/3
$UXC_{FT}-UXC_{ZS}$	+9.1	9	9/0	+9.2	10	10/0
$UXC_{FT}-W2V$	+17.5	9	9/0	+19.2	10	9/0
$UXC_{FT}-N2V$	−4.9	4	4/6	−3.3	3	3/6
N2V−W2V	+20.3	9	9/1	+16.2	9	9/1

N2V is significantly stronger than UXC_{FT} on both metrics for A.UML, C.Img, Lucene, Bash, HDF, and Chrome, whereas UXC_{FT} is significantly stronger on JabRef, SH-3D, and T.Mates; most of these differences have large effects. On HDC, UXC_{FT} is significantly stronger on macro-F1 with a small effect, and the micro-F1 difference is not significant.

Answer to RQ1.

Representations that encode dependencies, either directly through random walks (N2V) or through dependency-guided fine-tuning of UniXcoder (UXC_{FT}), provide the strongest mapping performance on most systems under MLP. N2V and UXC_{FT} do not dominate each other: N2V has the highest median and is significantly stronger than UXC_{FT} on six systems, whereas UXC_{FT} is significantly stronger on JabRef, SH-3D, and T.Mates. Fine-tuning consistently improves UniXcoder over its zero-shot representation and produces the best macro-F1 on two systems. W2V and UXC_{ZS}, which do not encode dependencies, differ from each other in both directions, and W2V is weaker than N2V and UXC_{FT} on most systems; W2V nevertheless remains stronger than N2V on JabRef. Thus, no single file representation performs best across all systems and metrics.

6.1 RQ2: Mapping Performance as File, Member, and Folder Context Are Added

RQ2 examines how mapping performance changes when graph structure is added to each representation. Figure 3 reports median macro-F1 and micro-F1 for the MLP and HGT under F, FM, FF, and FFM. For contextual representation, we use the fine-tuned variant UXC_{FT}. F adds message passing over file dependencies, FM additionally introduces member nodes, and FF and FFM introduce folder nodes.

Table 5 summarizes five steps. For each system, Δ is the difference between the two configuration medians over 100 runs. The table reports the median of these ten system-level differences, the number of systems with a positive difference, and the number of systems with a significant gain or loss.

File-level message passing. Moving from MLP to file-level dependency propagation helps W2V and UXC_{FT} more than N2V. For W2V and UXC_{FT}, F improves the median macro-F1 by 7.5 and 6.7 points and the median micro-F1 by 9.1 and 4.6 points, with positive differences on nine systems for each representation and metric. W2V has no significant loss, and UXC_{FT} has one, on HDC macro-F1. For N2V, the medians change by 0.8 and 2.2 points, with positive differences on six and seven systems. Significant losses occur on Lucene and HDC for both metrics and on HDF for macro-F1, but significant gains still outnumber them (five to three on macro-F1, seven to two on micro-F1).

Member-level context. Adding member nodes to F lowers the median macro-F1 by 12.2 points for W2V and by 1.2 points for both N2V and UXC_{FT}, and the median micro-F1 by 4.9, 1.3, and 1.8 points. The decline is largest for W2V, where all ten differences have large effects. Significant losses outnumber significant gains for all three representations: on macro-F1, seven systems decline significantly for W2V, five for N2V, and six for UXC_{FT}, against significant gains on three, none, and one. Adding member nodes also changes how features enter the model, depending on the representation (Section 4.2), so these results describe the evaluated member configuration and not member information in general.

Folder-level context. Adding folder nodes produces the largest and most consistent changes. FF improves over F, and FFM improves over FM, on both metrics for every system and representation. All 120 system–representation–metric comparisons are significant with a large effect. Across the three representations, the median macro-F1 gains range from 25.6 to 44.4 points for FF $-$ F and from 28.9 to 43.0 points for FFM $-$ FM. The corresponding micro-F1 gains range from 16.5 to 32.1 points and from 20.6 to 39.9 points. Each comparison improves in at least 96 of the 100 paired runs.

Within the folder-containing configurations, the effect of adding member nodes depends on the representation. FFM improves macro-F1 over FF on six systems each for N2V and UXC_{FT}, with median gains of 1.6 and 1.3 points, but on only three systems for W2V, with a median loss of 4.4 points. For micro-F1, it improves seven, seven, and three systems for N2V, UXC_{FT}, and W2V, with median changes of $+0.8$, $+1.1$, and -1.0 points. Significant macro-F1 losses occur on five systems for W2V and on JabRef, Bash, and Chrome for both N2V and UXC_{FT}. For micro-F1, N2V loses significantly only on Bash, and UXC_{FT} on no system.

Folder nodes and the directory baseline. The large gains from adding folder nodes show that directory organization is informative for the evaluated mappings. To examine whether these gains also help files that directory information alone does not place, we compare FM and FFM with UXC_{FT} features against the directory hierarchy baseline of Section 5.2.1. The baseline reaches 89–100% accuracy on every system, and its median number of errors per split is zero on C.Img, SH-3D, and T.Mates, and at most 27 on all other systems except Lucene (85.5) and Chrome (574).

The baseline is highly accurate on every system. Without folder nodes, FM falls well below it and gets many files wrong that the baseline gets right. With folder nodes, FFM comes close to the baseline on most systems, exceeds it only on Lucene, and matches it on SH-3D. Folder nodes therefore bring the model close to the information already available from the directory hierarchy.

This does not mean that FFM handles the files the baseline gets wrong. Among those files, FFM corrects more of FM's mistakes than it introduces on Lucene, HDF, A.UML, and T.Mates, but loses more correct predictions than it gains on JabRef, Bash, Chrome, and HDC; on C.Img and SH-3D the baseline makes no errors. JabRef shows why a higher overall accuracy is not sufficient: FFM improves over FM overall, yet it loses far more correct predictions than it gains on the files the baseline gets wrong. These counts are file and split events, so the same file can appear in several splits, and they compare FM with FFM, not each model with the baseline. They also do not show whether the model places files whose module differs from that of their directory.

Answer to RQ2.

The effect on mapping performance depends on what is added and on the representation. File-level propagation helps W2V and UXC_{FT} more than N2V, whose embeddings are already learned from file dependencies. Finer-grained member nodes give no consistent gain and often lower performance, most strongly for W2V. Folder nodes add the most, improving macro-F1 and micro-F1 on all ten systems for all three representations with significant large effects, but this largely reflects information already in the directory hierarchy: FFM comes close to a directory-only baseline, and where that baseline fails, folder nodes can turn correct FM predictions into incorrect ones. Strong folder-based scores therefore do not show that the model handles files whose directory location is insufficient or misleading.

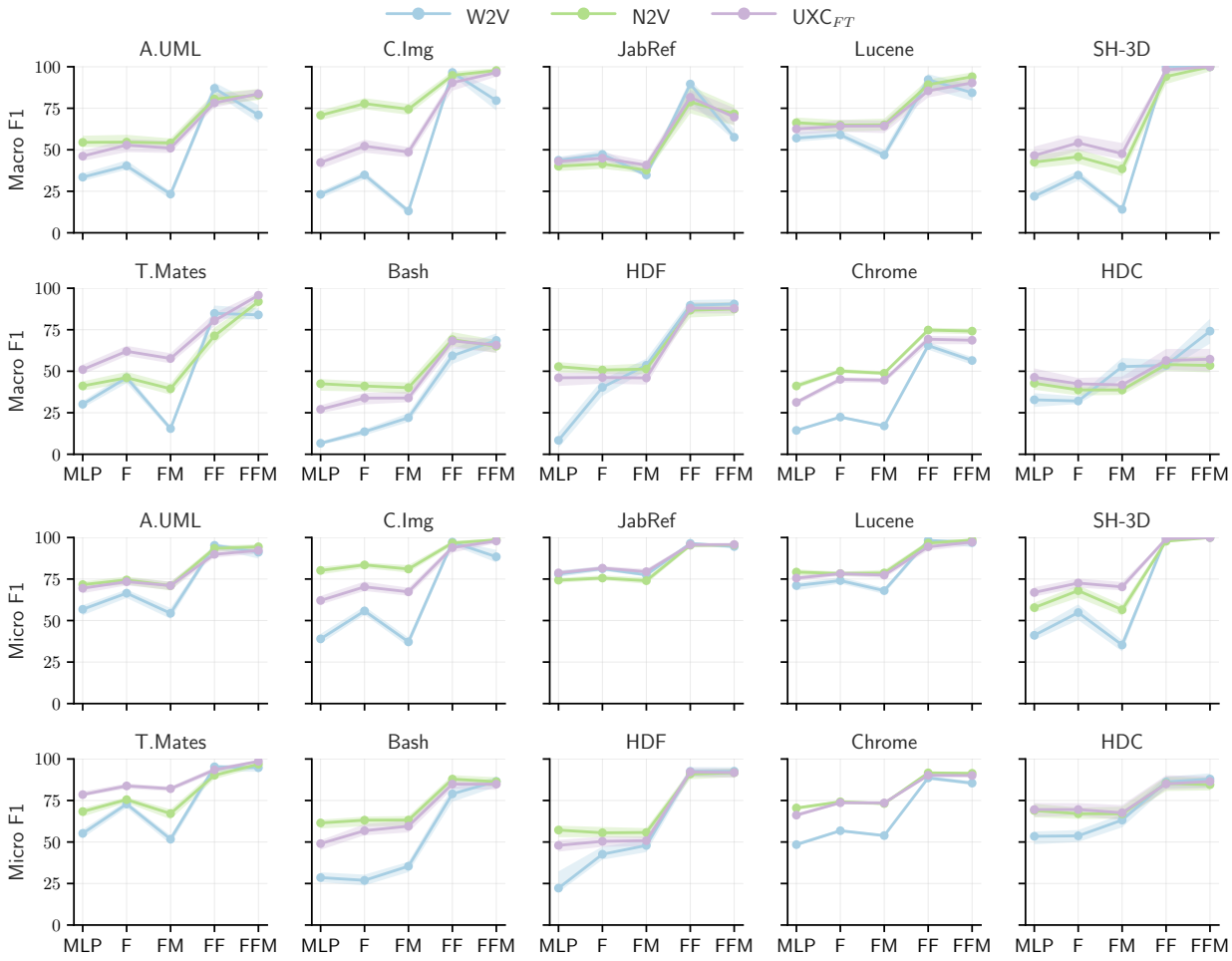


Fig. 3: Median macro-F1 (top) and micro-F1 (bottom) for each representation, per system, across the graph-free MLP and HGT with increasing graph structure (F, FM, FF, FFM). Shaded areas denote interquartile ranges across the 100 runs.

Table 5: Change in macro- and micro-F1 as graph structure is added, by representation. Each row compares the later configuration with the earlier one. Each cell gives the median over ten systems of the difference between configuration medians in F1 points, followed in parentheses by the number of systems with a positive difference and the number of significant positive/negative differences (paired comparison over the 100 runs, Holm-corrected). Shading marks the median difference: blue for gains, purple for losses, and darker for larger changes.

	W2V		N2V		UXC$_{FT}$	
Step	Macro F1	Micro F1	Macro F1	Micro F1	Macro F1	Micro F1
F − MLP	+7.5 (9; 9/0)	+9.0 (9; 8/0)	+0.8 (6; 5/3)	+2.2 (7; 7/2)	+6.7 (9; 8/1)	+4.6 (9; 9/0)
FM − F	−12.2 (3; 3/7)	−4.9 (3; 3/7)	−1.2 (2; 0/5)	−1.3 (3; 0/6)	−1.2 (2; 1/6)	−1.8 (2; 1/7)
FFM − FF	−4.4 (3; 2/5)	−1.0 (3; 2/5)	+1.6 (6; 5/3)	+0.8 (7; 5/1)	+1.3 (6; 5/3)	+1.1 (7; 5/0)
FF − F	+44.4 (10; 10/0)	+32.1 (10; 10/0)	+25.6 (10; 10/0)	+18.7 (10; 10/0)	+29.9 (10; 10/0)	+16.5 (10; 10/0)
FFM − FM	+43.0 (10; 10/0)	+39.9 (10; 10/0)	+28.9 (10; 10/0)	+22.3 (10; 10/0)	+32.3 (10; 10/0)	+20.6 (10; 10/0)

RQ3: Comparing Predictors and Dependency Direction

RQ2 shows that adding structure raises mapping performance but can also lower it, depending on the representation and system. Because RQ2 uses HGT only, RQ3 tests whether neighbor information also improves mapping with simpler predictors, namely GAT and a neighbor MLP ($\mathcal{N}_{\text{MLP}}$), and whether the direction in which neighbors are aggregated changes the result. We compare the $\mathcal{N}_{\text{MLP}}$, GAT, and HGT with neighbors off, on the directed graph, and on the undirected graph, for all four input representations.

Differences are paired *on* minus *off* changes in F1 points, and positive values favor neighbor information. The effect of direction is the paired undirected minus directed change,

and positive values favor undirected neighbors. We report the median over the 100 splits. With neighbors off, the $\mathcal{N}_{\text{MLP}}$ is the MLP of RQ1, and GAT and HGT receive no dependency messages.

How does the benefit of dependency neighbors vary across predictors and representations? Table 6 shows the change in F1 when neighbor information is switched on, for each predictor and input representation. With undirected neighbors, the median change is positive in every cell on both metrics. GAT and the $\mathcal{N}_{\text{MLP}}$ improve on at least eight systems in every cell and are never significantly lower, whereas HGT gains least with every representation and is significantly lower on a few systems with N2V and W2V. With directed neighbors, the median macro-F1 is lower with N2V for all three predictors, by 0.6 to 2.8 points, and significant losses occur on up to six systems. The other representations still gain with directed neighbors; only HGT loses by 0.2 to 1.6 macro-F1 points. For GAT and the $\mathcal{N}_{\text{MLP}}$, the gain from undirected neighbors is smallest with N2V ($+1.4$ and $+1.2$ macro-F1 points) and largest with UXC_{ZS} ($+8.2$ and $+7.7$).

This comparison differs from RQ2, where file-level message passing (F) is compared with the MLP, which changes the predictor together with the available structure. Here the predictor is fixed, and dependency neighbors are switched on and off, so a gain of F over the MLP does not imply that HGT benefits from dependency neighbors. With N2V, for example, F raises the median macro-F1 over the MLP, whereas HGT with directed neighbors is lower than HGT without neighbors.

Does the direction of neighbor aggregation matter? Undirected aggregation gives the higher median in almost every cell of Table 6, which raises the question of whether it is better on every system. Figure 4 compares the two directions per system. It is not: the change from directed to undirected aggregation is positive on only five to eight systems per cell, and significant changes occur in both directions. On macro-F1, SH-3D, T.Mates, and JabRef score higher with directed aggregation in all 12 cells, and C.Img in eight. Lucene, Bash, Chrome, and HDC score higher with undirected aggregation in all 12 cells, and A.UML and HDF in 11. The differences reach 14.6 points for undirected aggregation (Bash, W2V, $\mathcal{N}_{\text{MLP}}$) and 8.7 points for directed aggregation (T.Mates, W2V, $\mathcal{N}_{\text{MLP}}$). Micro-F1 gives the same pattern for seven systems, whereas JabRef, C.Img, and A.UML are mixed. Undirected neighbors also add the reverse edges, so these results do not separate the direction of propagation from the larger neighborhood.

Which predictor scores highest? Figure 5 pools the ten systems with undirected neighbors: each point is the median of one system and the boxes show the spread across systems. GAT has the highest median macro-F1 with every representation and the highest median micro-F1 with every representation except N2V, where GAT and HGT have the same median. Most of the variation lies between systems and not between splits: between-system differences account for 67 to 94% of the variance. The choice of predictor matters most for W2V and UXC_{ZS}, where the medians of the four methods span 12 to 14 points on both metrics, and least for N2V, where they span 7 points on macro-F1 and 4 on micro-F1; with UXC_{FT} they span 7 and 8 points. GAT also has the highest win rate across splits with every representation and on both metrics. All three predictors with neighborhood aggregation exceed the MLP without neighbors in every median comparison, and UXC_{FT} has the highest median macro-F1 for every predictor, with GAT and UXC_{FT} giving the highest scores overall.

Answer to RQ3.

Neighbor information raises the median score for all three predictors and all four input representations with undirected aggregation, and with directed aggregation everywhere except N2V, where the median macro-F1 is lower. The benefit depends on the predictor: GAT and the $\mathcal{N}_{\text{MLP}}$ gain reliably, HGT least, and GAT scores highest in most comparisons, although the differences between predictors are small relative to the variation across systems and matter most for W2V and UXC_{ZS}. It also depends on the representation, with larger gains for W2V and UXC_{ZS}, which do not encode dependencies, than for N2V and UXC_{FT}. The better direction follows the system and not the predictor or the representation: SH-3D and T.Mates favor directed aggregation, and Lucene, Bash, Chrome, HDC, and HDF favor undirected aggregation. With neighbors, UXC_{FT} gains more than N2V and has the highest median macro-F1 for every predictor, with comparable micro-F1.

7 Discussion

The research questions examine where architectural signal resides and how a mapping model can use it. RQ1 compares what file representations encode before message passing, RQ2 adds file, member, and folder context and compares the folder gains with a directory-based rule, and RQ3 examines how the predictor and the direction in which it aggregates dependency neighbors change what they contribute. This section considers the three together and states the conditions under which they hold.

Dependency information in file representations. RQ1 6 compares file representations before the predictor exchanges information between linked files, so the results reflect the

Table 6: Change in F1 when neighbor information is switched on, by input vector and graph condition. Each cell gives the median over ten systems of the paired within-split change in F1 points, followed in parentheses by the number of systems with a positive median change and the numbers of significant positive and negative changes (paired Wilcoxon, Holm). D: directed dependencies (source-to-target); UD: undirected; off: no neighbor information. Shading marks the median change: blue for gains, purple for losses, and darker for larger changes.

		N2V		W2V		UXC_{ZS}		UXC_{FT}	
Model	Contrast	Macro F1	Micro F1	Macro F1	Micro F1	Macro F1	Micro F1	Macro F1	Micro F1
HGT	D−off	−0.6 (4; 4/6)	+1.6 (6; 6/4)	+0.2 (5; 4/5)	+1.8 (7; 7/3)	+1.6 (9; 7/0)	+2.7 (10; 10/0)	+0.7 (7; 6/3)	+1.8 (8; 8/0)
	UD−off	+0.5 (8; 5/2)	+0.8 (9; 8/0)	+2.9 (7; 7/3)	+4.5 (8; 8/2)	+1.8 (10; 10/0)	+2.5 (10; 10/0)	+0.9 (10; 8/0)	+1.6 (10; 10/0)
	UD−D	+0.3 (6; 5/4)	+0.1 (5; 3/4)	+0.7 (7; 4/3)	+1.5 (6; 5/4)	0.0 (5; 4/3)	+0.1 (5; 5/4)	+0.6 (6; 5/3)	+0.2 (5; 4/4)
GAT	D−off	−2.8 (4; 3/6)	−0.8 (3; 2/5)	+1.5 (8; 6/2)	+2.4 (7; 7/1)	+3.8 (10; 9/0)	+6.2 (10; 10/0)	+0.7 (6; 4/2)	+2.2 (8; 7/1)
	UD−off	+1.4 (9; 8/0)	+1.4 (10; 10/0)	+5.6 (8; 7/0)	+6.7 (8; 8/0)	+8.2 (10; 10/0)	+9.5 (10; 10/0)	+3.8 (10; 8/0)	+4.1 (10; 10/0)
	UD−D	+3.8 (7; 6/2)	+2.2 (8; 8/1)	+1.9 (7; 5/3)	+1.6 (7; 5/3)	+3.3 (6; 6/4)	+2.2 (6; 6/2)	+3.0 (6; 6/4)	+1.3 (7; 6/1)
$\mathcal{N}_{MLP}$	D−off	−0.6 (4; 4/5)	+0.8 (6; 5/4)	+3.4 (9; 9/1)	+2.7 (9; 8/1)	+4.8 (8; 7/1)	+5.1 (9; 9/0)	+2.8 (7; 6/2)	+2.9 (7; 7/2)
	UD−off	+1.2 (10; 8/0)	+1.2 (9; 8/0)	+5.1 (10; 10/0)	+4.9 (10; 10/0)	+7.7 (10; 10/0)	+7.3 (10; 10/0)	+5.8 (10; 10/0)	+4.6 (10; 10/0)
	UD−D	+1.6 (6; 6/4)	+0.4 (6; 5/3)	+1.3 (6; 6/3)	+1.9 (6; 6/2)	+3.5 (6; 6/4)	+2.5 (6; 6/4)	+3.1 (6; 6/3)	+1.3 (7; 7/3)

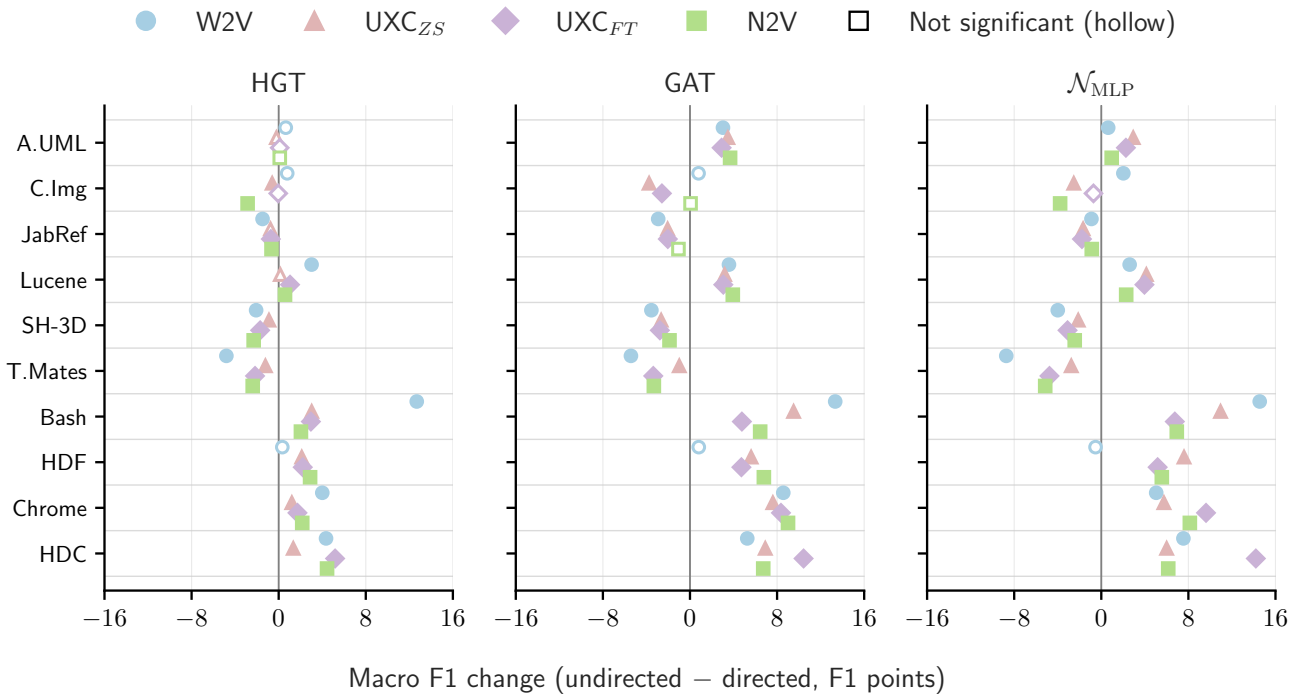


Fig. 4: Change in macro-F1, in F1 points, from directed to undirected aggregation (undirected − directed) for each predictor and input representation; negative values mean directed aggregation scored higher. Each point is the median over 100 paired splits; hollow markers are not significant.

architectural information encoded in each representation. Node2Vec, which is learned from the dependency graph alone, without labels, has the highest median micro-F1 and macro-F1 on six of the ten systems. Fine-tuned UniXcoder significantly improves over its zero-shot counterpart on micro-F1 on all ten systems and on macro-F1 on nine. Since the zero-shot encoder receives no dependency signal, this shows that fine-tuning on dependency-derived pairs adds information beyond that of pretrained code representations alone. Dependency information can therefore provide useful architectural signal before a predictor combines information across dependency links, either by encoding graph position directly in Node2Vec or by adapting a contextual code representation through dependency-guided fine-tuning.

The usefulness of this signal varies with the relation between dependency structure and the reference architecture. Node2Vec's micro-F1 is lower on the three least homophilic systems, SH-3D, HDF, and T.Mates (Table 2), with a median

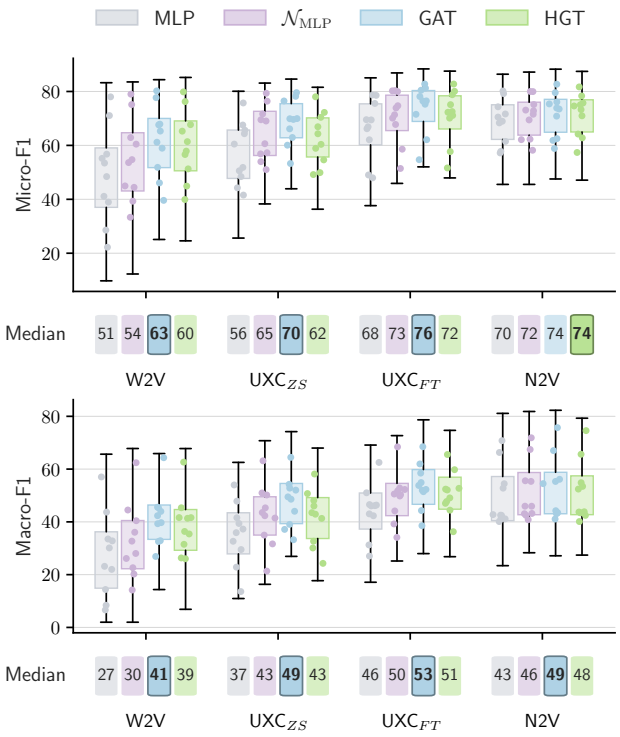


Fig. 5: Micro-F1 (top) and macro-F1 (bottom) of the graph-free MLP and of the $\mathcal{N}_{\text{MLP}}$, GAT, and HGT with undirected neighbors, for each input representation. Each point is one system (median over 100 splits), boxes show the spread across the ten systems, and the values below the boxes are medians across systems; bold marks the highest median in each group.

of 58 compared with 72 on the remaining seven. When dependencies frequently cross module boundaries, random walks may place files from different modules in similar structural neighborhoods, limiting the module-discriminating information available to Node2Vec. Homophily is computed from the reference mapping and is therefore unavailable before a mapping exists; it is used here to interpret the observed pattern, not to select a representation in practice.

Fine-tuned UniXcoder shows a different pattern on the low-homophily systems. Its median micro-F1 is 67 on SH-3D, HDF, and T.Mates, compared with 69.5 on the remaining seven. It exceeds Node2Vec by 9 and 10 micro-F1 points on SH-3D and T.Mates, which combine low homophily with the highest average degrees, $\langle k \rangle$ of 23.43 and 22.90. In these systems, dependency neighborhoods may span several modules, while the code content represented by UniXcoder can provide complementary evidence within such mixed neighborhoods. HDF is the exception: Node2Vec remains ahead of UXC$_{\text{FT}}$ (57 versus 48 micro-F1), although every representation performs comparatively poorly on this system.

The two representations without dependency information, Word2Vec and zero-shot UniXcoder, show no consistent ordering. Their weaker overall results reinforce that the advantage of Node2Vec and UXC$_{\text{FT}}$ is associated with dependency information rather than with the use of a graph embedding or a code encoder alone.

The RQ2 and RQ3 comparison concerns a different use of dependencies. While Node2Vec learns file vectors from unlabeled random walks, the neighbor-based predictors use dependency links while learning the mapping from the available labels. The results show that useful dependency information in an input representation does not by itself determine whether this additional neighbor use helps: with N2V, directed aggregation lowers the median macro-F1 for all three predictors, whereas undirected aggregation raises it, and HGT gains least from neighbors with every representation. The effect therefore depends on how the predictor aggregates the neighborhood and in which direction.

Folder organization and architectural exceptions. The gains from adding folder nodes in RQ2 show that directory organization helps reproduce the reference mapping. However, extending a directory assignment is different from identifying files that depart from it. The directory analysis separates these cases by checking, on the files that a directory-based baseline gets wrong, which incorrect predictions of FM the folder nodes correct and which correct predictions they lose (Section 6.1).

Lucene and HDF mainly illustrate the first case. Their directories are pure under the reference mapping, and directory-baseline errors occur where the immediate directory has no labeled examples. Correcting these errors shows that a model can recover an assignment when local supervision is missing. It gives little evidence about conflicts between directory structure and architecture. Bash has mixed directories and therefore presents a different problem. JabRef shows why the distinction matters: adding folder context improves overall accuracy while losing more correct predictions on directory-baseline errors than it gains.

A mapping tool intended to identify architectural exceptions should therefore be evaluated on files that depart from their directory assignment, alongside overall accuracy. A rule-based directory baseline using only the initial labels provides a useful reference, but the counts compare FM with FFM on the baseline's errors and do not show how each model compares with the baseline on those files. Recovering assignments in directories without initial labels is useful, but does not establish that a model can identify files whose module assignment conflicts with their directory.

Dependency neighborhood processing and aggregation direction. RQ3 distinguishes dependency information encoded in file representations from its additional use during prediction. With undirected aggregation, dependency neigh-

bors raise the median score for GAT, HGT, and the neighbor $\mathcal{N}_{\text{MLP}}$ with every representation, including N2V and UXC_{FT}, so dependency-informed initialization does not make neighbor access redundant. The gains are larger for W2V and UXC_{ZS}, which do not encode dependencies, than for N2V and UXC_{FT}, which reinforces the RQ1 finding that dependencies provide signal beyond file content. HGT gains least with every representation and GAT scores highest in most comparisons, although the differences between predictors are small relative to the variation across systems. GAT and HGT differ in their attention formulation and parameterization, so the comparison does not isolate why GAT performs better. These findings concern the evaluated file-level predictors, not the usefulness of dependency types in general or the folder- and member-aware configurations.

The preferred direction of aggregation follows the system and not the predictor or the representation. SH-3D and T.Mates favor directed aggregation, and Lucene, Bash, Chrome, HDC, and HDF favor undirected aggregation, on both metrics. Undirected aggregation also adds the reverse edges, so the comparison does not isolate the direction of propagation from the larger neighborhood. With N2V, directed aggregation lowers the median macro-F1 for all three predictors. The N2V vectors were learned on the undirected dependency graph, which may make directed aggregation less consistent with the neighborhood that each vector encodes, but we did not test this.

Connectivity and neighbor information. Connectivity describes how much neighboring context is available, measured here by the average node degree, while homophily describes whether linked files share a module. Shallow message passing reaches only as far as its number of layers, so files with few neighbors provide little to aggregate, whereas Node2Vec can encode more distant context through random walks. Connectivity can therefore constrain additional message passing even when the initial structural representation remains useful.

Several results are consistent with this. With N2V and undirected aggregation, no predictor changes significantly on Bash and HDC, the two systems with the lowest average degree. T.Mates and SH-3D combine low homophily with among the highest average degrees, gain most from file-level message passing with N2V in RQ2, and favor directed aggregation in RQ3. Across systems, the advantage of undirected over directed aggregation decreases with the expected number of cross-module neighbors per file, $\langle k \rangle (1 - H)$, more clearly than with homophily alone, although HDF, which has low homophily but a much lower average degree, does not follow this pattern. One reading is that reverse edges supply missing context for files with few neighbors but add mostly cross-module neighbors where files already have many. With ten systems and correlated descriptors, we cannot separate connectivity from homophily, system size, or label scarcity, and we did not test this.

Implications for evaluating mapping models. Evaluation should establish what the input representation already provides, then isolate neighbor access within a fixed predictor. Comparing a graph model with a graph-free MLP changes both the predictor and its access to structure; switching neighbors on and off within the same predictor tests whether that access helps.

Macro and micro F1 scores should be read together, since correctly mapping more files does not necessarily improve performance across modules. The directory analysis adds a further requirement: aggregate scores should be accompanied by evidence about files that depart from directory-based assignments. Together, these comparisons distinguish the contributions of input features, neighbor processing, and neighbor direction, while exposing limitations hidden by aggregate scores. These points concern the ten systems and the compared configurations and do not rank the signals or the predictors in general.

8 Discussion

The research questions examine where architectural signal resides and how a mapping model can use it. RQ1 compares what file representations encode before message passing, RQ2 adds file, member, and folder context and compares the folder gains with a directory-based rule, and RQ3 examines how the predictor and the direction in which it aggregates dependency neighbors change what they contribute. This section considers the three together and states the conditions under which they hold.

Dependency information in file representations. RQ1 6 compares file representations before the predictor exchanges information between linked files, so the results reflect the architectural information encoded in each representation. Node2Vec, which is learned from the dependency graph alone, without labels, has the highest median micro-F1 and macro-F1 on six of the ten systems. Fine-tuned UniXcoder significantly improves over its zero-shot counterpart on micro-F1 on all ten systems and on macro-F1 on nine. Since the zero-shot encoder receives no dependency signal, this shows that fine-tuning on dependency-derived pairs adds information beyond that of pretrained code representations alone. Dependency information can therefore provide useful architectural signal before a predictor combines information across dependency links, either by encoding graph position directly in Node2Vec or by adapting a contextual code representation through dependency-guided fine-tuning.

The usefulness of this signal varies with the relation between dependency structure and the reference architecture. Node2Vec's micro-F1 is lower on the three least homophilic systems, SH-3D, HDF, and T.Mates (Table 2), with a median of 58 compared with 72 on the remaining seven. When depen-

dencies frequently cross module boundaries, random walks may place files from different modules in similar structural neighborhoods, limiting the module-discriminating information available to Node2Vec. Homophily is computed from the reference mapping and is therefore unavailable before a mapping exists; it is used here to interpret the observed pattern, not to select a representation in practice.

Fine-tuned UniXcoder shows a different pattern on the low-homophily systems. Its median micro-F1 is 67 on SH-3D, HDF, and T.Mates, compared with 69.5 on the remaining seven. It exceeds Node2Vec by 9 and 10 micro-F1 points on SH-3D and T.Mates, which combine low homophily with the highest average degrees, $\langle k \rangle$ of 23.43 and 22.90. In these systems, dependency neighborhoods may span several modules, while the code content represented by UniXcoder can provide complementary evidence within such mixed neighborhoods. HDF is the exception: Node2Vec remains ahead of UXC_{FT} (57 versus 48 micro-F1), although every representation performs comparatively poorly on this system.

The two representations without dependency information, Word2Vec and zero-shot UniXcoder, show no consistent ordering. Their weaker overall results reinforce that the advantage of Node2Vec and UXC_{FT} is associated with dependency information rather than with the use of a graph embedding or a code encoder alone.

The RQ2 and RQ3 comparison concerns a different use of dependencies. While Node2Vec learns file vectors from unlabeled random walks, the neighbor-based predictors use dependency links while learning the mapping from the available labels. The results show that useful dependency information in an input representation does not by itself determine whether this additional neighbor use helps: with N2V, directed aggregation lowers the median macro-F1 for all three predictors, whereas undirected aggregation raises it, and HGT gains least from neighbors with every representation. The effect therefore depends on how the predictor aggregates the neighborhood and in which direction.

Folder organization and architectural exceptions. The gains from adding folder nodes in RQ2 show that directory organization helps reproduce the reference mapping. However, extending a directory assignment is different from identifying files that depart from it. The directory analysis separates these cases by checking, on the files that a directory-based baseline gets wrong, which incorrect predictions of FM the folder nodes correct and which correct predictions they lose (Section 6.1).

Lucene and HDF mainly illustrate the first case. Their directories are pure under the reference mapping, and directory-baseline errors occur where the immediate directory has no labeled examples. Correcting these errors shows that a model can recover an assignment when local supervision is missing. It gives little evidence about conflicts between directory structure and architecture. Bash has mixed directories and therefore presents a different problem. JabRef shows why the distinction matters: adding folder context improves overall accuracy while losing more correct predictions on directory-baseline errors than it gains.

A mapping tool intended to identify architectural exceptions should therefore be evaluated on files that depart from their directory assignment, alongside overall accuracy. A rule-based directory baseline using only the initial labels provides a useful reference, but the counts compare FM with FFM on the baseline's errors and do not show how each model compares with the baseline on those files. Recovering assignments in directories without initial labels is useful, but does not establish that a model can identify files whose module assignment conflicts with their directory.

Dependency neighborhood processing and aggregation direction. RQ3 distinguishes dependency information encoded in file representations from its additional use during prediction. With undirected aggregation, dependency neighbors raise the median score for GAT, HGT, and the neighbor $\mathcal{N}_{\text{MLP}}$ with every representation, including N2V and UXC_{FT}, so dependency-informed initialization does not make neighbor access redundant. The gains are larger for W2V and UXC_{ZS}, which do not encode dependencies, than for N2V and UXC_{FT}, which reinforces the RQ1 finding that dependencies provide signal beyond file content. HGT gains least with every representation and GAT scores highest in most comparisons, although the differences between predictors are small relative to the variation across systems. GAT and HGT differ in their attention formulation and parameterization, so the comparison does not isolate why GAT performs better. These findings concern the evaluated file-level predictors, not the usefulness of dependency types in general or the folder- and member-aware configurations.

The preferred direction of aggregation follows the system and not the predictor or the representation. SH-3D and T.Mates favor directed aggregation, and Lucene, Bash, Chrome, HDC, and HDF favor undirected aggregation, on both metrics. Undirected aggregation also adds the reverse edges, so the comparison does not isolate the direction of propagation from the larger neighborhood. With N2V, directed aggregation lowers the median macro-F1 for all three predictors. The N2V vectors were learned on the undirected dependency graph, which may make directed aggregation less consistent with the neighborhood that each vector encodes, but we did not test this.

Connectivity and neighbor information. Connectivity describes how much neighboring context is available, while homophily describes whether linked files share a module. Connectivity matters particularly for shallow message passing, whose reach is limited by the number of layers. Sparse neighborhoods restrict the features available for aggregation, whereas Node2Vec can encode more distant context

through random walks. Connectivity can therefore constrain additional message passing even when the initial structural representation remains useful.

With N2V and undirected aggregation, no predictor changes significantly on Bash and HDC, the systems with the lowest average degree. This is consistent with limited local context reducing the benefit of neighbor aggregation, but does not establish connectivity as the cause. In contrast, T.Mates and SH-3D have low homophily but among the highest average degrees, and they are the systems with the largest N2V micro-F1 gains from file-level message passing in RQ2 ($+7.2$ and $+10.3$ points) and the systems that favor directed aggregation in RQ3. HDF has similarly low homophily but a much lower degree and no comparable gain. These observations suggest that connectivity is relevant to the context available for shallow message passing, but with ten systems and correlated descriptors we cannot separate connectivity from homophily, system size, or label scarcity, and we did not test this.

Implications for evaluating mapping models. Evaluation should establish what the input representation already provides, then isolate neighbor access within a fixed predictor. Comparing a graph model with a graph-free MLP changes both the predictor and its access to structure; switching neighbors on and off within the same predictor tests whether that access helps.

Macro and micro F1 scores should be read together, since correctly mapping more files does not necessarily improve performance across modules. The directory analysis adds a further requirement: aggregate scores should be accompanied by evidence about files that depart from directory-based assignments. Together, these comparisons distinguish the contributions of input features, neighbor processing, and neighbor direction, while exposing limitations hidden by aggregate scores. These points concern the ten systems and the compared configurations and do not rank the signals or the predictors in general.

9 Conclusion

This study examines which implementation-level signals, including code text, static dependencies, folder organization, and member-level information, carry architectural information and how their representation and use affect code-to-architecture mapping on ten systems. We encode file information using lexical Word2Vec, structural Node2Vec, and contextual UniXcoder in zero-shot (UXC_{ZS}) and dependency-guided fine-tuned (UXC_{FT}) variants, and evaluate these representations with MLP- and GNN-based predictors. Representations that encode dependencies, Node2Vec and UXC_{FT}, outperform Word2Vec on most systems when evaluated with the MLP, whereas Word2Vec and UXC_{ZS}, which do not, differ from each other in both directions. Dependency-guided fine-tuning significantly improves mapping performance over UXC_{ZS} on at least nine of ten systems, although Node2Vec and UXC_{FT} are each significantly stronger on different systems and no representation is best on every system. Folder nodes produce the largest and most consistent gains within HGT variants, but largely recover information already available to a directory rule using the initial labels and do not consistently improve the files on which that rule fails. Member nodes do not give consistent gains, and their effect cannot generally be separated from changes in feature construction and placement.

The controlled experiments distinguish dependency information encoded in file representations from its additional use during prediction by keeping the input representations fixed and switching dependency neighbors off, directed, or undirected for three predictors on all ten systems. With undirected aggregation, dependency neighbors raise the median score for GAT, HGT, and the neighbor $\mathcal{N}_{\text{MLP}}$, which concatenates own-file features with mean-neighbor features, across all four representations, with larger gains for Word2Vec and UXC_{ZS} than for Node2Vec and UXC_{FT}; with directed aggregation, the median macro-F1 is lower only with Node2Vec. These results show that dependency links provide useful information beyond lexical and pretrained code features, while gains with Node2Vec and UXC_{FT} show that neighbor access can also complement representations that already encode dependencies. How much neighborhood information helps depends on the predictor and on the direction of aggregation: HGT gains least with every representation, GAT scores highest in most comparisons, and the better direction depends on the system, with SH-3D and T.Mates favoring directed aggregation and Lucene, Bash, Chrome, HDC, and HDF favoring undirected aggregation. Undirected aggregation also adds the reverse edges, so the experiments do not separate the direction of propagation from the larger neighborhood.

Dependencies carry architectural signal beyond file content, and a mapping model can use it in two ways: by encoding it in the file representation, as Node2Vec and dependency-guided fine-tuning do, or by giving the predictor access to direct neighbors. Both help, but the second depends on the predictor and on the direction in which neighbors are aggregated. Evaluating a mapping model should therefore establish what its input already encodes, isolate neighbor access within a fixed predictor, and examine the files that depart from their directory assignment. These findings concern ten systems and the compared configurations, and do not rank the signals or the predictors in general.

References

1. Andritsos P, Tsaparas P, Miller RJ, Sevcik KC (2004) Limbo: Scalable clustering of categorical data. In: International Conference

on Extending Database Technology, Springer, pp 123–146, DOI 10.1007/978-3-540-24741-8_9
2. Anquetil N, Lethbridge TC (1999) Recovering software architecture from the names of source files. Journal of Software Maintenance: Research and Practice 11(3):201–221, DOI 10.1002/(sici)1096-908x(199905/06)11:3<201::aid-smr192>3.0.co;2-1
3. Bibi M, Maqbool O, Kanwal J (2016) Supervised learning for orphan adoption problem in software architecture recovery. Malaysian Journal of Computer Science 29(4):287–313
4. Bittencourt RA, Santos GJdS, Guerrero DDS, Murphy GC (2010) Improving automated mapping in reflexion models using information retrieval techniques. In: 17th Working Conference on Reverse Engineering, pp 163–172
5. Brunsfeld M, contributors (2018) Tree-sitter: An incremental parsing system for programming tools. https://tree-sitter.github.io/tree-sitter/, accessed: 2026-04-22
6. Chen J, Wang Z, Jiang Y, Pang J, Zhang T, Pan M, Sun J (2022) NEGAR: network embedding guided architecture recovery for software systems. In: 2022 29th Asia-Pacific Software Engineering Conference (APSEC), IEEE, pp 367–376, DOI 10.1109/apsec57359.2022.00049
7. Christl A, Koschke R, Storey MA (2005) Equipping the reflexion method with automated clustering. In: 12th Working Conference on Reverse Engineering (WCRE'05), IEEE, pp 10–pp
8. Christl A, Koschke R, Storey MA (2007) Automated clustering to support the reflexion method. Information and Software Technology 49(3):255–274
9. Clements P, Garlan D, Little R, Nord R, Stafford J (2003) Documenting software architectures: views and beyond. In: 25th International Conference on Software Engineering, IEEE, pp 740–741, DOI 10.1109/icse.2003.1201264
10. Cliff N (1993) Dominance statistics: Ordinal analyses to answer ordinal questions. Psychological bulletin 114(3):494, DOI 10.1037/0033-2909.114.3.494
11. Corazza A, Di Martino S, Maggio V, Scanniello G (2011) Investigating the use of lexical information for software system clustering. In: 2011 15th European Conference on Software Maintenance and Reengineering, IEEE, pp 35–44, DOI 10.1109/csmr.2011.8
12. Florean A, Jalal L, Sinkala ZT, Herold S (2021) A comparison of machine learning-based text classifiers for mapping source code to architectural modules. In: 15th European Conference on Software Architecture
13. Fuchß D, Liu H, Hey T, Keim J, Koziolek A (2025) Enabling architecture traceability by llm-based architecture component name extraction. In: 2025 IEEE 22nd International Conference on Software Architecture (ICSA), IEEE, pp 1–12
14. Garcia J, Popescu D, Mattmann C, Medvidovic N, Cai Y (2011) Enhancing architectural recovery using concerns. In: 2011 26th IEEE/ACM International Conference on Automated Software Engineering (ASE 2011), IEEE, pp 552–555, DOI 10.1109/ase.2011.6100123
15. Garcia J, Krka I, Mattmann C, Medvidovic N (2013) Obtaining ground-truth software architectures. In: 35th International Conference on Software Engineering (ICSE), IEEE, pp 901–910, DOI 10.1109/icse.2013.6606639
16. Grover A, Leskovec J (2016) node2vec: Scalable feature learning for networks. In: Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining, pp 855–864, DOI 10.1145/2939672.2939754
17. Guo D, Lu S, Duan N, Wang Y, Zhou M, Yin J (2022) Unixcoder: Unified cross-modal pre-training for code representation. In: Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp 7212–7225
18. Harman M, Yao X (2010) Software module clustering as a multi-objective search problem. IEEE Transactions on Software Engineering 37(2):264–282, DOI 10.1109/tse.2010.26
19. He K, Zhang X, Ren S, Sun J (2016) Deep residual learning for image recognition. In: Proceedings of the IEEE conference on computer vision and pattern recognition, pp 770–778
20. Hu Z, Dong Y, Wang K, Sun Y (2020) Heterogeneous graph transformer. In: Proceedings of the Web Conference 2020, pp 2704–2710
21. Jabeen R, Ericsson M, Nordqvist J, Wingkvist A (2025) Graph convolution networks for mapping source code entities to architectural modules. In: 2025 IEEE 22nd International Conference on Software Architecture Companion (ICSA-C), IEEE, pp 472–481, DOI 10.1109/icsa-c65153.2025.00071
22. Jabeen R, Ericsson M, Nordqvist J, Wingkvist A (2026) GAER: Graph auto-encoders for unsupervised software architecture recovery. Proceedings of the ACM on Software Engineering 3(FSE), DOI 10.1145/3808187
23. Jabeen R, Ericsson M, Nordqvist J, Wingkvist A (2026) Graph embeddings for software architecture recovery. Nordic Machine Intelligence DOI 10.5617/nmi.13153, URL https://journals.uio.no/NMI/article/view/13153
24. Jabeen R, Ericsson M, Nordqvist J, Wingkvist A (2026) Graph representation learning for software architecture recovery. In: ESANN
25. Johansson N, Caporuscio M (2025) Mapping source code to software architecture using embedding models: Insights from an analysis of data properties. SSRN Electronic Journal URL https://ssrn.com/abstract=5798927, preprint, SSRN 5798927
26. Johansson N, Caporuscio M, Olsson T (2024) Mapping source code to software architecture by leveraging large language models. In: European Conference on Software Architecture, Springer, pp 133–149, DOI 10.1007/978-3-031-70797-1_8
27. Kargar M, Isazadeh A, Izadkhah H (2017) Semantic-based software clustering using hill climbing. In: 2017 International Symposium on Computer Science and Software Engineering Conference (CSSE), IEEE, pp 55–60, DOI 10.1109/csicsse.2017.8320117
28. Lenhard J, Blom M, Herold S (2019) Exploring the suitability of source code metrics for indicating architectural inconsistencies. Software Quality Journal 27(1):241–274
29. Lutellier T, Chollak D, Garcia J, Tan L, Rayside D, Medvidović N, Kroeger R (2017) Measuring the impact of code dependencies on software architecture recovery techniques. IEEE Transactions on Software Engineering 44(2):159–181, DOI 10.1109/tse.2017.2671865
30. Mancoridis S, Mitchell BS, Chen Y, Gansner ER (1999) Bunch: A clustering tool for the recovery and maintenance of software system structures. In: Proceedings IEEE International Conference on Software Maintenance (ICSM'99), IEEE, pp 50–59, DOI 10.1109/icsm.1999.792498
31. Maqbool O, Babri HA (2004) The weighted combined algorithm: A linkage algorithm for software clustering. In: Eighth European Conference on Software Maintenance and Reengineering (CSMR 2004), IEEE, pp 15–24, DOI 10.1109/csmr.2004.1281402
32. Mikolov T, Sutskever I, Chen K, Corrado GS, Dean J (2013) Distributed representations of words and phrases and their compositionality. Advances in Neural Information Processing Systems 26
33. Misra J, Annervaz K, Kaulgud V, Sengupta S, Titus G (2012) Software clustering: Unifying syntactic and semantic features. In: 2012 19th Working Conference on Reverse Engineering, IEEE, pp 113–122, DOI 10.1109/wcre.2012.21
34. Mitchell BS, Mancoridis S (2006) On the automatic modularization of software systems using the bunch tool. IEEE Transactions on Software Engineering 32(3):193–208, DOI 10.1109/tse.2006.31
35. Mkaouer W, Kessentini M, Shaout A, Koligheu P, Bechikh S, Deb K, Ouni A (2015) Many-objective software remodularization using nsga-iii. ACM Transactions on Software Engineering and Methodology 24(3):1–45, DOI 10.1145/2729974

36. Mohammadi S, Izadkhah H (2019) A new algorithm for software clustering considering the knowledge of dependency between artifacts in the source code. Information and Software Technology 105:252–256, DOI 10.1016/j.infsof.2018.09.001
37. Murphy GC, Notkin D, Sullivan K (1995) Software reflexion models: Bridging the gap between source and high-level models. In: 3rd ACM SIGSOFT Symposium on Foundations of Software Engineering, pp 18–28
38. Nair V, Hinton GE (2010) Rectified linear units improve restricted boltzmann machines. In: Proceedings of the 27th International Conference on Machine Learning (ICML), Omnipress, pp 807–814, URL `https://www.cs.toronto.edu/~fritz/absps/reluICML.pdf`
39. Naseem R, Maqbool O, Muhammad S (2013) Cooperative clustering for software modularization. Journal of Systems and Software 86(8):2045–2062, DOI 10.1016/j.jss.2013.03.080
40. Naseem R, Deris MM, Maqbool O, Shahzad S (2019) Euclidean space based hierarchical clusterers combinations: an application to software clustering. Cluster Computing 22(Suppl 3):7287–7311, DOI 10.1007/s10586-017-1408-0
41. Nguyen HV (2023) Depends: Multilingual dependency analysis tool. `https://github.com/multilang-depends/depends`, accessed: 2025-08-17
42. Olsson T, Ericsson M, Wingkvist A (2019) Semi-automatic mapping of source code using naive bayes. In: Proceedings of the 13th European Conference on Software Architecture, pp 209–216
43. Olsson T, Ericsson M, Wingkvist A (2021) s4rdm3x: A tool suite to explore code to architecture mapping techniques. Journal of Open Source Software 6(58)
44. Olsson T, Ericsson M, Wingkvist A (2022) To automatically map source code entities to architectural modules with naive bayes. Journal of Systems and Software 183:111095, DOI 10.1016/j.jss.2021.111095
45. van den Oord A, Li Y, Vinyals O (2018) Representation learning with contrastive predictive coding. arXiv preprint arXiv:180703748
46. Papachristou M (2019) Software clusterings with vector semantics and the call graph. In: Proceedings of the 27th ACM Joint ESEC/FSE, pp 1184–1186, DOI 10.1145/3338906.3342483
47. Parnas DL (1972) On the criteria to be used in decomposing systems into modules. Communications of the ACM 15(12):1053–1058
48. Parsa S, Bushehrian O (2005) A new encoding scheme and a framework to investigate genetic clustering algorithms. Journal of Research and Practice in Information Technology 37(1):127–143
49. Pourasghar B, Izadkhah H, Isazadeh A, Lotfi S (2021) A graph-based clustering algorithm for software systems modularization. Information and Software Technology 133:106469, DOI 10.1016/j.infsof.2020.106469
50. Puchala SPR, Chhabra JK, Rathee A (2022) Ensemble clustering based approach for software architecture recovery. International Journal of Information Technology 14(4):2013–2019, DOI 10.1007/s41870-021-00846-0
51. Rathee A, Chhabra JK (2017) Software remodularization by estimating structural and conceptual relations among classes and using hierarchical clustering. In: Advanced Informatics for Computing Research: ICAICR 2017, Springer, pp 94–106, DOI 10.1007/978-981-10-5780-9_9
52. Richards M, Ford N (2025) Fundamentals of Software Architecture: A Modern Engineering Approach. O'Reilly Media
53. Risi M, Scanniello G, Tortora G (2012) Using fold-in and fold-out in the architecture recovery of software systems. Formal Aspects of Computing 24(3):307–330, DOI 10.1007/s00165-011-0199-y
54. Rukmono SA, Ochoa L, Chaudron M (2024) Deductive software architecture recovery via chain-of-thought prompting. In: 44th International Conference on Software Engineering: New Ideas and Emerging Results, pp 92–96, DOI 10.1145/3639476.3639776
55. Sadat Jalali N, Izadkhah H, Lotfi S (2019) Multi-objective search-based software modularization: Structural and non-structural features. Soft Computing 23(21), DOI 10.1007/s00500-018-3666-z
56. Sinkala ZT, Herold S (2021) Inmap: Automated interactive code-to-architecture mapping recommendations. In: 18th International Conference on Software Architecture, pp 173–183
57. Sinkala ZT, Herold S (2023) An integrated approach to package and class code-to-architecture mapping using inmap. In: 2023 IEEE 20th International Conference on Software Architecture (ICSA), IEEE, pp 164–174
58. Teymourian N, Izadkhah H, Isazadeh A (2020) A fast clustering algorithm for modularization of large-scale software systems. IEEE Transactions on Software Engineering 48(4):1451–1462, DOI 10.1109/tse.2020.3022212
59. Tzerpos V, Holt RC (1997) The orphan adoption problem in architecture maintenance. In: Working Conference on Reverse Engineering, IEEE, pp 76–82
60. Veličković P, Cucurull G, Casanova A, Romero A, Lio P, Bengio Y (2017) Graph attention networks. arXiv preprint arXiv:171010903 DOI 10.48550/arXiv.1710.10903
61. Wilcoxon F (1945) Individual comparisons by ranking methods. Biometrics bulletin 1(6):80–83, DOI 10.2307/3001968
62. Yang K, Wang J, Fang Z, Wu P, Song Z (2022) Enhancing software modularization via semantic outliers filtration and label propagation. Information and Software Technology 145:106818, DOI 10.1016/j.infsof.2021.106818
63. Zhang Y, Xu Z, Liu C, Chen H, Sun J, Qiu D, Liu Y (2023) Software architecture recovery with information fusion. In: Proceedings of the 31st ACM Joint ESEC/FSE, pp 1535–1547, DOI 10.1145/3611643.3616285